

Reporte: Letterbooks

Martínez Cruz Lizandro

Álvarez Molina Luana Sofía
Soto Mortera David

Peréz Peréz Iker Daniel

Junio 2026

Resumen

Desarrollo de una aplicación web que permite registrar libros, publicar reseñas con citas textuales, comentar, calificar y gestionar perfiles de usuario. El sistema se implementó con una arquitectura cliente-servidor: frontend en React, backend REST con Spring Boot y persistencia en MariaDB. Autenticación basada en JWT y separación entre entidades de dominio y DTOs.

1. Introducción

El objetivo de este proyecto es desarrollar **Letterbooks**, una plataforma web orientada a lectores que desean compartir opiniones sobre libros, descubrir nuevas lecturas y construir una comunidad en torno a la literatura.

El sistema exige una arquitectura web completa con interfaz accesible desde el navegador, API REST, base de datos relacional y mecanismos de autenticación. El usuario puede registrarse, iniciar sesión, explorar un catálogo de libros, escribir reseñas con citas destacadas, comentar publicaciones ajenas, dar *likes* y personalizar su perfil con avatar, banner y preferencias.

2. Análisis

2.1. Objetivos

- Diseñar e implementar un esquema relacional que modele usuarios, libros, reseñas, citas, comentarios e interacciones sociales.
- Desarrollar un backend REST con Spring Boot que encapsule la lógica de negocio y el acceso a datos.
- Construir una interfaz web en React que permita navegar, buscar libros y gestionar contenido de forma intuitiva.
- Implementar autenticación segura mediante tokens JWT y contraseñas hasheadas con BCrypt.
- Permitir la carga de imágenes portadas, avatares y banners.
- Aplicar pruebas unitarias e de integración.

2.2. Requerimientos funcionales

Entre las funcionalidades principales se incluyen:

- Registro e inicio de sesión de usuarios.
- CRUD de libros con metadatos (título, sinopsis, autor, género, editorial, ISBN, año, páginas).
- Publicación de reseñas con calificación numérica y texto.
- Inclusión de citas textuales asociadas a una reseña.
- Comentarios sobre reseñas y sistema de likes sobre reseñas, citas y comentarios.
- Perfil de usuario con biografía, imagen de avatar, banner y autor, genero y libro favorito.
- Visualización de libros populares y reseñas recientes en la página de aterrizaje.

3. Diseño

3.1. Arquitectura general

El sistema fue diseñado con una arquitectura en capas que separa responsabilidades entre el cliente web, el servidor de aplicaciones y la base de datos.

- **Frontend (React + Vite):** Gestiona rutas, formularios, consumo de la API y renderizado de componentes. Organizado en `paginas/`, `componentes/` y `api/`.
- **Backend (Spring Boot):** Expone endpoints REST, valida peticiones, ejecuta reglas de negocio y persiste datos.
- **Base de datos (MariaDB):** Persistencia definida en `scripts/esquema.sql`.

La comunicación entre frontend y backend se realiza exclusivamente mediante HTTP y JSON. Las rutas protegidas requieren un token JWT en el encabezado `Authorization`.

3.2. Patrones de diseño

Durante el desarrollo se emplearon varios patrones de forma explícita:

- **Repository (Spring Data JPA):** Cada entidad cuenta con una interfaz de repositorio (`LibroRepositorio`, `ResenaRepositorio`, etc.) que abstrae las consultas SQL y permite operaciones CRUD declarativas.
- **Service Layer:** La lógica reside en clases `@Service` (`LibroServicio`, `ResenaServicio`, `AutenticacionServicio`), evitando que los controladores accedan directamente a la base de datos.
- **DTO (Data Transfer Object):** Objetos como `RegistroLibro`, `VistaLibro` y `NuevaResena` transportan datos entre capas sin exponer entidades JPA completas ni campos sensibles.
- **Filter Chain (Spring Security):** `FiltroAcceso` Intercepta peticiones HTTP, valida el JWT y establece el contexto de autenticación antes de llegar a los controladores.
- **MVC en el frontend:** las páginas actúan como vistas, los módulos `api/` como capa de acceso a datos remotos y los componentes gestionan la interacción del usuario.

3.3. Modelo de datos

El esquema relacional incluye tablas independientes (**genero**, **autor**, **editorial**), entidades centrales (**usuario**, **libro**, **perfil**, **resena**) y tablas de interacción (**cita**, **comentario**, **likes_resena**, **likes_cita**, **likes_comentario**, **calificacion_resena**). Las llaves foráneas garantizan integridad. Las restricciones **UNIQUE** en tablas de likes evitan duplicados por usuario.

En el backend, cada tabla se mapea a una entidad JPA en el paquete **modelo**. Las relaciones **@ManyToOne** y **@OneToMany** reflejan cardinalidades del esquema SQL. Por ejemplo, **Resena** mantiene una lista de **Cita** con **cascade = CascadeType.ALL** y métodos auxiliares **agregarCita()** y **removeCita()** para sincronizar la relación bidireccional.

3.4. Diagrama de clases

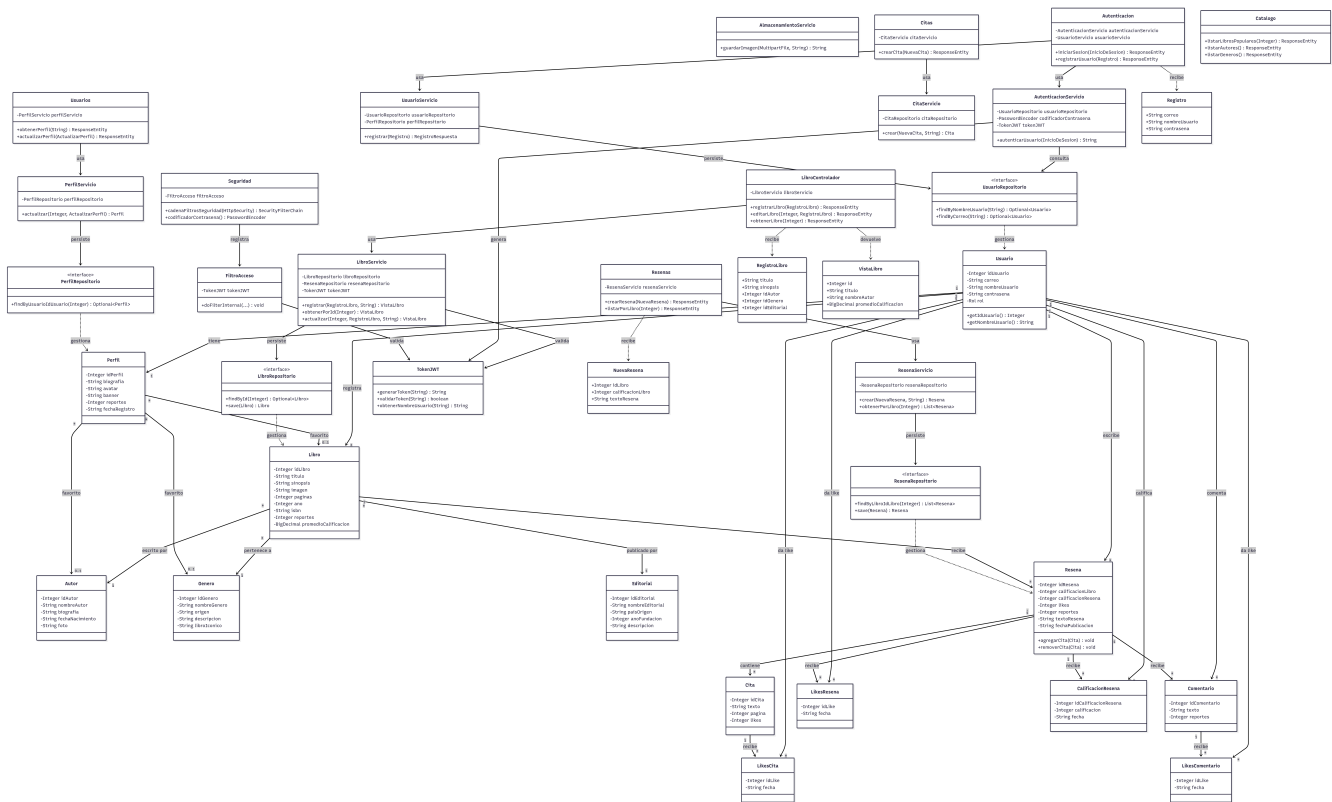


Figura 1: Diagrama de clases del sistema Letterbooks.

El diagrama de la Figura 1 resume la estructura del backend y su relación con las entidades de dominio. Se distinguen:

- **Capa de dominio:** Usuario, Perfil, Libro, Resena, Cita, Comentario y entidades auxiliares.
- **Capa de persistencia:** repositorios JPA que implementan el acceso a datos.
- **Capa de servicios:** clases que coordinan validaciones, transformaciones a DTO y reglas de negocio.
- **Capa de controladores:** endpoints REST expuestos al frontend.
- **Seguridad:** TokenJWT, FiltroAcceso y configuración de Seguridad.

4. Implementación

4.1. Backend

- `modelo/`: Entidades JPA.
- `repositorio/`: Interfaces Spring Data.
- `servicio/`: Lógica de negocio.
- `controlador/`: Endpoints REST.
- `dto/`: Objetos de transferencia.
- `configuracion/`: CORS, seguridad y datos iniciales.
- `seguridad/`: Generación y validación de JWT.

```
@Entity
@Table(name = "libro")
public class Libro {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_libro")
    private Integer idLibro;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_autor")
    private Autor autor;

    @NotBlank(message = "El título no puede estar vacío")
    @Column(name = "titulo", nullable = false)
    private String titulo;
    // ...
}
```

El servicio `LibroServicio` valida la existencia de autor, género y editorial antes de persistir, obtiene al usuario autenticado a partir del token JWT y transforma el resultado en un `VistaLibro` para la respuesta HTTP.

La autenticación se implementó en `AutenticacionServicio`: las contraseñas se almacenan hasheadas con BCrypt y, después de un login exitoso, se genera un token JWT firmado que el frontend almacena en el contexto de sesión. La clase `Seguridad` define qué rutas son públicas (registro, login, catálogo de libros populares, listado de reseñas por libro) y cuáles requieren autenticación.

4.2. Frontend

La interfaz se implementó con React, Vite y Tailwind CSS. React Router define las rutas principales: página de aterrizaje, login, registro, biblioteca, detalle de libro, perfil propio y perfil público.

```
export async function obtenerLibrosPopulares(limite = 3) {
    const respuesta = await fetch(
        `${URL_BASE}/catalogo/libros/populares?limite=${limite}`
    );
    if (!respuesta.ok) {
        throw new Error('Error al obtener los libros populares');
    }
}
```

```

    }
    return await respuesta.json();
}

```

4.3. Infraestructura y despliegue

El archivo `docker-compose.yml`:

- **db**: MariaDB con volumen persistente y *healthcheck*.
- **backend**: Contenedor con el JAR de Spring Boot en el puerto 8080.
- **frontend**: Build de Vite servido con nginx en el puerto 80.

Hibernate actualiza el esquema en tiempo de ejecución (`ddl-auto: update`) y la clase `DatosIniciales` inserta libros, usuarios y reseñas de demostración para facilitar pruebas y la presentación del sistema.

4.4. Pruebas

Se implementaron pruebas unitarias e de integración en el backend utilizando JUnit 5, Spring Boot Test y H2 como base de datos en memoria para los tests. Entre los casos cubiertos se encuentran autenticación, repositorios de usuario y reseña, servicios de almacenamiento y controladores REST.

5. Elección de tecnologías

5.1. Backend: Java y Spring Boot

Se eligió Java por ser el lenguaje en el que nos sentiamos mas comodis. Spring Boot simplifica la configuración de un servidor, la inyección de dependencias y la integración con JPA, Security y Validation en un solo framework [6].

5.2. Frontend: React y Vite

React permite construir interfaces modulares mediante componentes reutilizables y gestionar el estado de la aplicación de forma declarativa [4]. Vite ofrece tiempos de compilación rápidos durante el desarrollo [5]. Tailwind CSS agiliza el diseño responsivo sin dejar de lado los archivos CSS para componentes complejos [3].

5.3. Base de datos: MariaDB

MariaDB fue seleccionada por que Luana menciona que ya habia trabajado con ella y confiamos en ella. El esquema SQL se diseñó manualmente y se fue modificando conforme necesitabamos y los cambios lo requerian.

5.4. Autenticación: JWT

Los JSON Web Tokens permiten implementar autenticación *stateless* en una API REST: el servidor no mantiene sesiones en memoria y el cliente envía el token en cada petición protegida [1]. La biblioteca `jjwt` se utilizó para firmar y validar tokens en el backend.

6. Dificultades

- **Coordinación en equipo:** Los merge fueron un desastre al principio, y el querer conectar tantas ideas de una manera que no rompiera el proyecto fue complicado pero se logro, tambien fue personalmente mi primera vez usando git de una manera seria, teniamos nuestra forma de trabajar para que se entendiera pero de todos modos fue complicado.
- **Relaciones JPA:** Mapear correctamente relaciones bidireccionales y evitar referencias circulares en serialización JSON demandó el uso de `@JsonIgnore` y DTOs.
- **Seguridad:** Configurar Spring Security para permitir rutas públicas específicas sin desproteger el resto de la API.
- **CORS y despliegue:** Al ejecutar frontend y backend en contenedores distintos, fue necesario configurar CORS y asegurar que las URLs de la API fueran accesibles desde el navegador.
- **Gestión del tiempo:** La implementación de likes, comentarios, citas resultó más tardada de lo que teniamos pensado.

7. Resultados

El sistema desarrollado permite registrar usuarios, explorar un catálogo de libros, publicar reseñas con citas, interactuar mediante comentarios y *likes*, y personalizar perfiles de lectores. A continuación se muestran capturas de la interfaz web en funcionamiento.

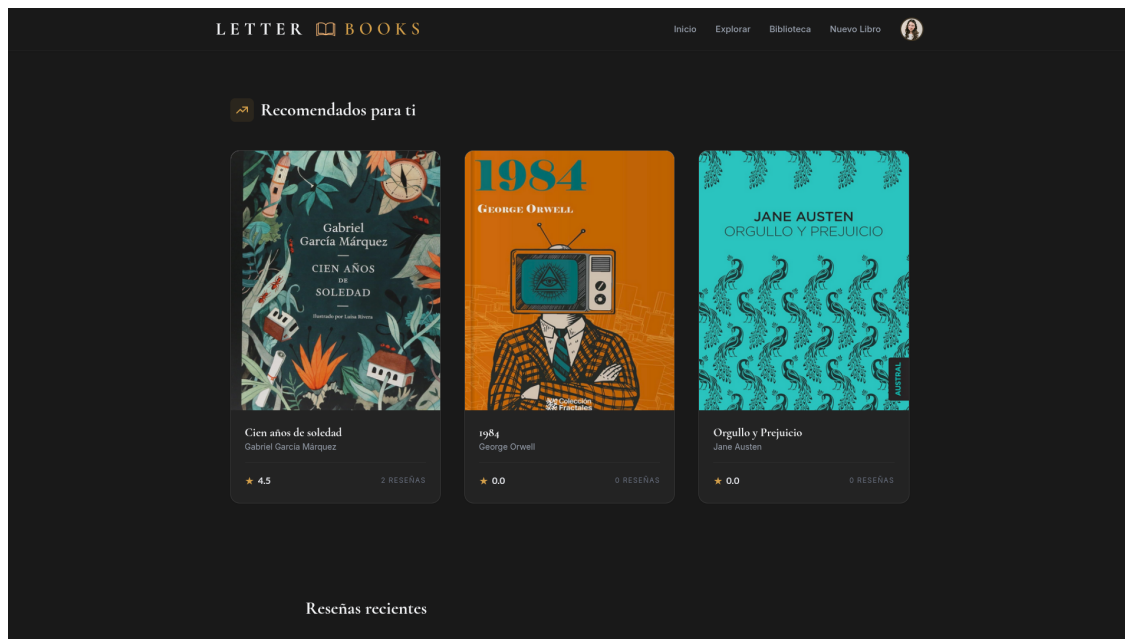


Figura 2: Página de aterrizaje con libros populares y reseñas recientes.

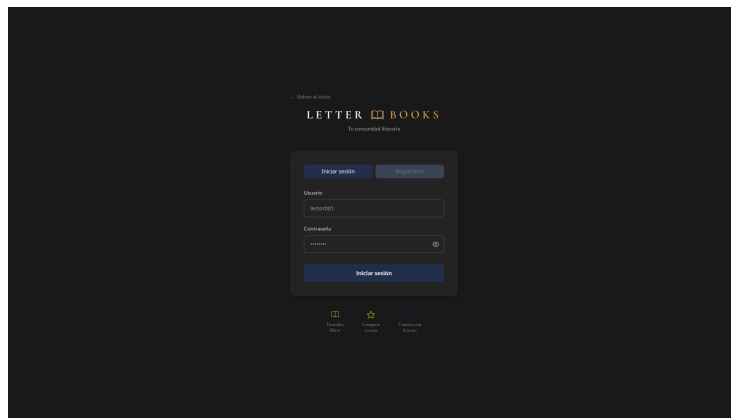


Figura 3: Pantalla de inicio de sesión y registro.

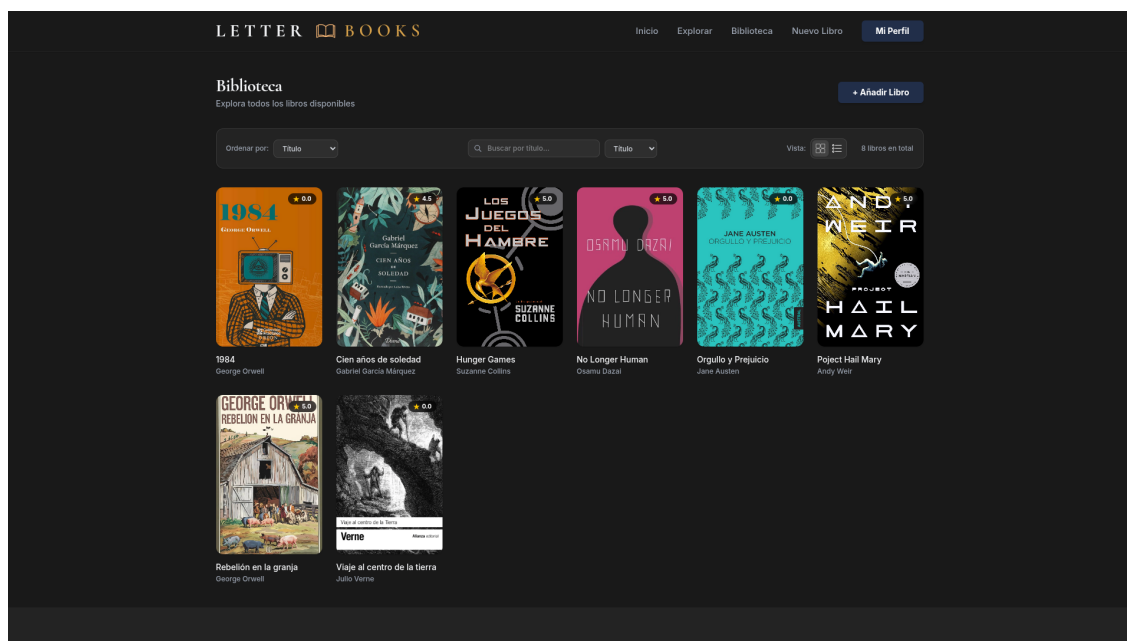


Figura 4: Vista de biblioteca con búsqueda, ordenamiento y catálogo de libros.

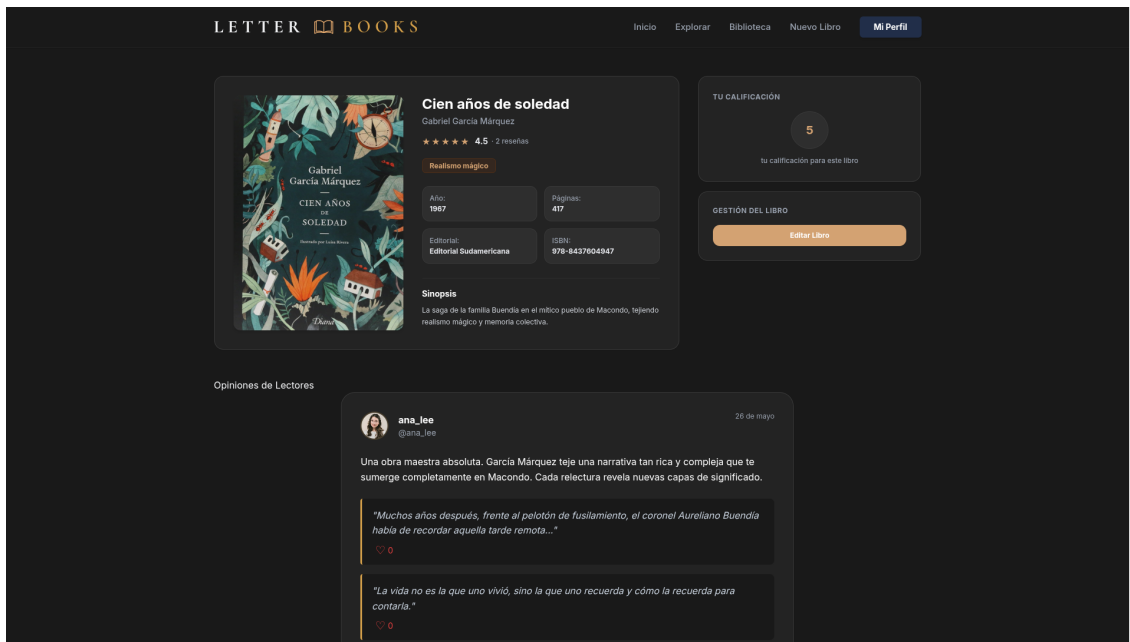


Figura 5: Detalle de un libro con metadatos, reseñas y citas textuales.

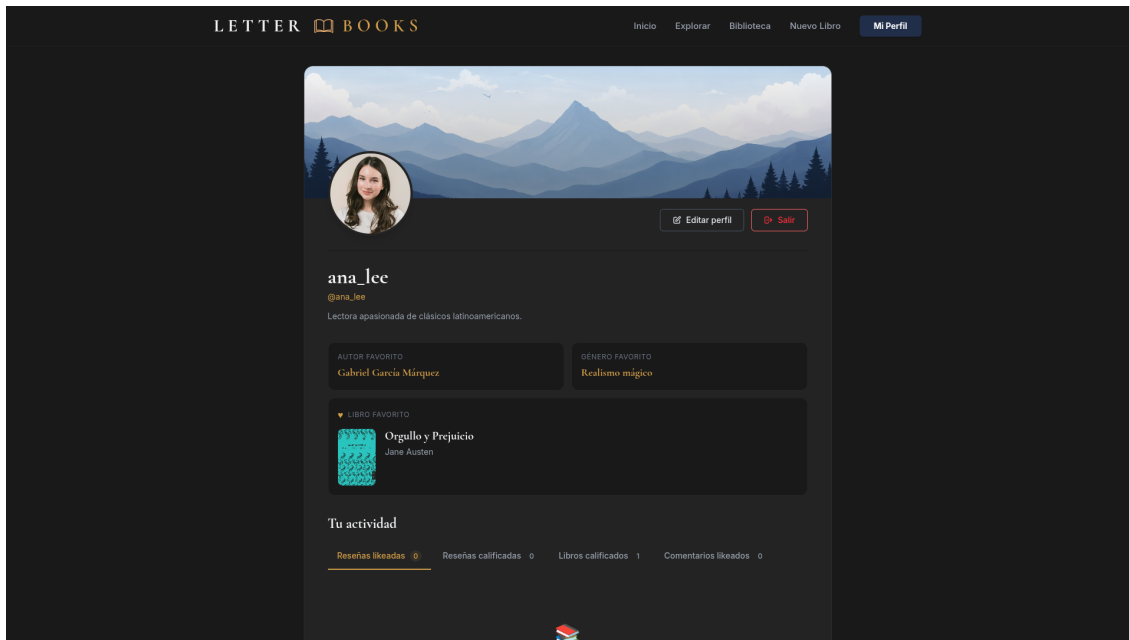


Figura 6: Perfil de usuario con preferencias literarias y actividad reciente.

Los resultados obtenidos demuestran que la arquitectura en capas permite integrar de forma coherente el frontend, el backend y la base de datos.

8. Conclusiones

Fue un desarrollo complicado por que la mayoría de nosotros no teníamos experiencia en proyectos de programación colaborativos, fue hablar incesantemente sobre que parte le tocaba a cada quien y como dividir los elementos faltantes equitativamente. Fue un aprendizaje importante para la comunicación en equipo, ya que gracias a eso no tuvimos muchos problemas a la hora de realizar los planes deseados.

Aprender a utilizar los frameworks fue otra maravilla de esto, la gente en internet ya creo la mayoría de la tecnología, si no es que toda, que usamos en este proyecto. No tuvimos que crear nada desde cero, alguien ya habia pensado en eso, solo es ver como adaptar esa solución a nuestro problema. El uso de frameworks facilitó tanto nuestro trabajo para que no sufrieramos inventado, alguien mas ya lo hizo, por que no aprovecharlo.

Nos dimos cuenta de la poderosa herramienta que es git, es tan poderosa que te deja dispararte a ti mismo en el pie, con los merge conflict y el hecho de que si no tienes integración continua, puedes subir cualquier porquería de código al repo. Dejando eso de lado, git es la única razón de que pudimos completarlo, fue eficiente ya que al dividirnos las partes que teníamos que hacer solo hacíamos una rama personal y hacíamos merge en la de trabajo para asegurarnos que funciona todo junto.

Como trabajo futuro se queda pendiente el hecho de los reportes y que existan moderadores, simplemente no nos dio la vida para implementar todo eso, pero sentimos que hicimos un buen trabajo y nos sentimos orgullosos de nuestro proyecto.

9. Referencias

Referencias

- [1] Auth0. Json web token (jwt). <https://jwt.io/introduction>, 2026.
- [2] Docker Inc. Docker compose documentation. <https://docs.docker.com/compose/>, 2026.
- [3] Tailwind Labs. Tailwind css documentation. <https://tailwindcss.com/docs>, 2026.
- [4] Meta. React documentation. <https://react.dev/>, 2026.
- [5] Vite Team. Vite guide. <https://vite.dev/guide/>, 2026.
- [6] VMware. Spring boot reference documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>, 2026.
- [7] VMware. Spring data jpa reference documentation. <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>, 2026.